

Applying the Brain’s Dual-Library Mechanism to Transformer Architectures

Heiko Wagner

Independent Researcher, Bonn, Germany
heikowagner@t-online.de

June 2026

Abstract

Current large language models (LLMs) concatenate system instructions, historical context, and factual data into the same token sequence. Through successive layers of self-attention, these distinct inputs intertwine—a monolithic blending that creates structural vulnerabilities: context drift, prompt injection, and the “lost in the middle” phenomenon. A landmark single-neuron recording study by Bausch et al. (2026) [4] suggests a biological alternative: human memory maintains a functional separation between *what* occurs (content) and the framework within which it occurs (context). This paper proposes a dual-stream Transformer that mirrors this separation. Isolated content and context streams, coupled through unidirectional cross-attention with a content-driven sigmoidal gate, yield three concrete benefits: mathematically guaranteed prompt injection resistance (proved by induction over transformer layers), mitigation of lost-in-the-middle degradation, and improved zero-shot generalisation through context-invariant representations. We validate the architecture across three experiments spanning proof-of-concept language modelling at 6.7M parameters, full-scale contrastive training with two pre-trained models (DeepSeek-Coder 6.7B + Llama 3.2 3B, 31.5M trainable gate parameters), and a conscience experiment that uses the architecture’s separation guarantee to give a coding agent ethical constraints that cannot be overridden by user input. The dual-stream Transformer achieves a $1.69\times$ improvement in validation loss over a parameter-matched monolithic baseline at the smaller scale, GC = 99.96% at full scale, and 95% overall accuracy on the conscience benchmark.

1 Introduction

In May 2023, Geoffrey Hinton resigned from Google to speak publicly about the risks of artificial intelligence [10]. In interviews with the *New York Times* and CBS, he described concerns about AI systems developing sub-goals misaligned with their programmers’ intentions and called for urgent research into how to control systems that may surpass human intelligence. Hinton, together with Sam Altman, Demis Hassabis, and hundreds of other researchers, signed the Center for AI Safety statement: “Mitigating the risk of extinction from AI should be a global priority alongside other societal-scale risks such as pandemics and nuclear war” [5].

Current approaches to AI safety rely on training. Reinforcement Learning from Human Feedback (RLHF) fine-tunes models on human preference data to produce helpful and harmless responses [2]. Constitutional AI, developed by Anthropic, trains models to self-critique and revise their outputs against a list of rules [3]. Both methods are statistical: they teach the model a correlation between certain input patterns and refusal behaviour. A creative enough prompt breaks that correlation. Every safety-trained model released since 2022 has been jailbroken within weeks.

Transformer-based large language models [12] process all input tokens through a unified self-attention mechanism. System instructions, conversation history, and factual data are concatenated into a single

sequence—through successive layers, tokens representing entirely different semantic functions freely attend to one another. This design, while remarkably effective for next-token prediction, introduces structural vulnerabilities that scale with context length.

Three problems are well documented. First, **prompt injection**: an adversarial user input can override system instructions because all tokens share the same attention space [11]. Current defences rely on training data quality and reinforcement learning from human feedback (RLHF), both of which are inherently adversarial—a determined attacker can always find prompts outside the training distribution. System-level approaches such as CaMeL [6] extract control and data flows from the trusted query to prevent untrusted content from hijacking the agent’s program flow. However, CaMeL does not modify the model architecture itself: the underlying transformer still blends content and context in a shared attention space, leaving context drift and lost-in-the-middle degradation unaddressed, and the model’s internal reasoning about untrusted data remains vulnerable to manipulation even when the agent’s external actions are constrained. Second, the **“lost in the middle”** phenomenon [9]: as context windows grow to millions of tokens, models increasingly overlook information placed in the centre of the input because attention weights dilute across an undifferentiated token pool. Third, **context drift**: long context windows degrade the precision of content representations because contextual tokens mask factual tokens.

A recent single-neuron recording study from the University of Bonn [4] suggests a radically different approach. Bausch et al. recorded 3,109 single neurons in the human medial temporal lobe and identified two largely distinct neuronal populations: stimulus neurons (invariant to task context) and context neurons (invariant to stimulus identity). Only 1.61% of neurons showed significant stimulus–context interaction. Crucially, the connection between the two populations is asymmetric and directional: firing of stimulus neurons in the entorhinal cortex predicted firing of context neurons in the hippocampus after approximately 40 ms, but not the reverse.

This paper translates these findings into a concrete Transformer architecture and validates it at scale. The contributions are:

1. A **dual-stream Transformer** with separate weight matrices for content and context streams, coupled through unidirectional cross-attention with a content-driven sigmoidal gate.
2. A **mathematical proof** by induction over transformer layers that the content stream has no computational path into the context stream ($\partial H_{\text{ctx}}/\partial x_c = 0$), establishing a structural guarantee against prompt injection.
3. A **proof-of-concept experiment** on a 1,750-sample corpus across 10 task categories, showing $1.69\times$ better validation loss and substantially reduced overfitting compared to a parameter-matched monolithic baseline.
4. A **full-scale adapter** that couples two pre-trained models (DeepSeek-Coder 6.7B + Llama 3.2 3B) through a 31.5M-parameter bottleneck gate, validated by a contrastive training experiment demonstrating $\text{GC} = 99.96\%$.
5. A **conscience experiment** that bakes ethical constraints into the context stream’s weights and routes declared intent through the architecturally protected context channel, achieving 95% overall accuracy across 20 test cases.

2 Biological Paradigm

Bausch et al. recorded 3,109 single neurons in the amygdala, parahippocampal cortex, entorhinal cortex, and hippocampus while patients performed a comparison task: pairs of pictures appeared on screen, and a question shown at the start of each trial specified the comparison rule—“Which is bigger?”, “Which

did you see more recently in real life?”, *“Which is brighter?”*. The question defined the task context; the pictures were the content.

The study identified two largely distinct neuronal populations in the medial temporal lobe:

- **Stimulus neurons** (597 identified)—fire selectively to specific picture content, regardless of which question context is active. Most (88%) are invariant to context.
- **Context neurons** (200 identified)—fire selectively to a particular task context (question), regardless of which picture is shown. Most (63.5%) are invariant to stimulus identity.

Across all 3,109 neurons, only 50 (1.61%) showed significant stimulus–context interaction—i.e., responded to specific picture–question combinations. Instead, separate orthogonal populations represent content and context independently, combining them via co-activation and reinstatement.

The critical architectural insight comes from the temporal dynamics: firing of stimulus neurons in the entorhinal cortex predicted firing of context neurons in the hippocampus after approximately 40 ms—**but not the reverse**. Context neurons showed increased excitability after pre-activation by their preferred question—a gating mechanism where prior context exposure guides which hippocampal context representations are reinstated when a stimulus is subsequently encountered. This directional asymmetry and gating behaviour are the key properties this paper translates directly into a transformer layer.

3 Dual-Stream Transformer Architecture

In standard Transformer models [12], query Q , key K , and value V matrices are computed across the entire token sequence simultaneously:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

If a prompt contains a factual statement followed by an instruction, the token embeddings for both elements modify each other in every layer. Context and content share a vector space; a long context window degrades the precision of the content representation.

We propose to give the model two separate token sequences rather than one. Instead of prepending a system prompt to the user message and hoping the model treats them differently, the architecture routes them through distinct processing paths from the embedding layer onward:

- **Content stream**—user input, documents, code, factual data (analogous to stimulus neurons).
- **Context stream**—system instructions, persona, output constraints (analogous to context neurons).

Each stream has its own weight matrices and is never concatenated with the other.

3.1 Gatekeeper Cross-Attention

To replicate the brain’s ability to reconstruct a complete memory from its parts, the architecture introduces an asymmetric cross-attention layer. The processed content acts as the driver (query), while the context serves as the lookup library (key and value):

$$H_{\text{cross}} = \text{Attention}(Q_{\text{content}}, K_{\text{context}}, V_{\text{context}}) \quad (2)$$

This design ensures that the context cannot overwrite the content; instead, the content actively retrieves the specific structural instructions needed to format or manipulate its data.

The final step requires a mathematical gate to control information flow, mimicking the 40 ms temporal delay observed in the Bonn study. Instead of adding the cross-attention output directly to the residual stream, the model utilises a sigmoid-controlled linear unit. Let H_{content} be the hidden states of the content stream. The gate activation G is defined as:

$$G = \sigma(W_g \cdot H_{\text{content}} + b_g) \quad (3)$$

The final integrated representation H_{final} is computed via an element-wise product:

$$H_{\text{final}} = \text{LayerNorm}(H_{\text{content}} + (G \odot H_{\text{cross}})) \quad (4)$$

Intuitively, the gate learns to answer a simple question for each hidden dimension: *does this content need external context to be processed correctly?* Since G is computed from the content stream itself ($W_g \cdot H_{\text{content}}$), the model conditions its reliance on context based on what the content already communicates.

- **Gate open** ($G \rightarrow 1$): The content is raw, unstructured data—“*The system peaked at 180 °C during stress testing.*” These tokens carry no inherent task signal. The gate allows retrieved context (e.g., a system prompt like “Extract metrics as JSON”) to shape the output.
- **Gate closed** ($G \rightarrow 0$): The content already implies the task—“*What is the capital of France?*” or “*SELECT * FROM users WHERE active = 1;*”. The tokens encode their own structural intent. Injecting additional context would be redundant or conflicting, so the gate suppresses it.

This conditional routing is structurally what prevents prompt injection: tokens inside the content stream that mimic instructions are evaluated strictly as content, and the gate—having learned from training that such patterns never originate in the context path—stays closed for them.

3.2 PyTorch Reference Implementation

The following defines a `ContentContextLayer` that implements parallel routing, cross-attention lookup, and the asymmetric gating mechanism:

```

1 import torch
2 import torch.nn as nn
3
4 class ContentContextLayer(nn.Module):
5     def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1):
6         super().__init__()
7         # 1. Isolated Self-Attention (Separate Neural Libraries)
8         self.content_self_attn = nn.MultiheadAttention(
9             d_model, nhead, dropout=dropout, batch_first=True)
10        self.context_self_attn = nn.MultiheadAttention(
11            d_model, nhead, dropout=dropout, batch_first=True)
12        self.norm1_content = nn.LayerNorm(d_model)
13        self.norm1_context = nn.LayerNorm(d_model)
14        # 2. Specialized Feed-Forward Networks
15        self.ffn_content = nn.Sequential(
16            nn.Linear(d_model, dim_feedforward), nn.GELU(),
17            nn.Dropout(dropout), nn.Linear(dim_feedforward, d_model))
18        self.ffn_context = nn.Sequential(
19            nn.Linear(d_model, dim_feedforward), nn.GELU(),
20            nn.Dropout(dropout), nn.Linear(dim_feedforward, d_model))
21        self.norm2_content = nn.LayerNorm(d_model)
22        self.norm2_context = nn.LayerNorm(d_model)

```

```

23     # 3. Gatekeeper and Pattern Completion
24     self.cross_attn = nn.MultiheadAttention(
25         d_model, nhead, dropout=dropout, batch_first=True)
26     self.gate_proj = nn.Linear(d_model, d_model)
27     self.norm_final = nn.LayerNorm(d_model)
28     self.dropout = nn.Dropout(dropout)
29
30     def forward(self, content_x, context_x,
31                 content_causal_mask=None, content_padding_mask=None,
32                 context_mask=None):
33         # Phase 1: Isolated processing paths
34         c_attn_out, _ = self.content_self_attn(
35             content_x, content_x, content_x,
36             attn_mask=content_causal_mask,
37             key_padding_mask=content_padding_mask)
38         content_x = self.norm1_content(content_x + self.dropout(c_attn_out))
39
40         ctx_attn_out, _ = self.context_self_attn(
41             context_x, context_x, context_x,
42             key_padding_mask=context_mask)
43         context_x = self.norm1_context(context_x + self.dropout(
44             ctx_attn_out))
45         # Phase 2: Feed-Forward updates
46         content_x = self.norm2_content(
47             content_x + self.dropout(self.ffn_content(content_x)))
48         context_x = self.norm2_context(
49             context_x + self.dropout(self.ffn_context(context_x)))
50         # Phase 3: Gatekeeper Mechanism
51         retrieved_context, _ = self.cross_attn(
52             query=content_x, key=context_x, value=context_x,
53             key_padding_mask=context_mask)
54         gate = torch.sigmoid(self.gate_proj(content_x))
55         gated_context = gate * retrieved_context
56         final_output = self.norm_final(
57             content_x + self.dropout(gated_context))
58         return final_output, content_x, context_x

```

Listing 1: ContentContextLayer implementing dual-stream processing with gatekeeper cross-attention.

3.3 Adapter Architecture at Scale

The reference implementation trains both streams from scratch. To validate the design at realistic scale without prohibitive compute, the architecture adapts to two different pre-trained models rather than training from scratch:

- **Content stream:** DeepSeek-Coder 6.7B Instruct (4-bit quantized, frozen). Hidden dimension 4096. Processes code, tool outputs, user requests.
- **Context stream:** Llama 3.2 3B Instruct (4-bit quantized, frozen). Hidden dimension 3072. Processes system instructions, ethical rules, declared intent.
- **Cross-attention gate:** 31.5M trainable parameters. Projects context from 3072 to 4096 dimensions, computes cross-attention with content as query, applies a bottleneck sigmoid gate ($4096 \rightarrow 256 \rightarrow 4096$), and adds the gated result to the content residual stream.

The gate formula follows Equation (4) but introduces a bottleneck that compresses the content hidden state before producing the per-dimension mixing coefficient:

$$G = \sigma(W_{\text{up}} \cdot \text{GELU}(W_{\text{down}} \cdot H_{\text{content}})) \quad (5)$$

with $W_{\text{down}} \in \mathbb{R}^{4096 \times 256}$ and $W_{\text{up}} \in \mathbb{R}^{256 \times 4096}$. The gate compresses the 4096-dimensional content hidden state to 256 dimensions, applies a GELU activation, projects back to 4096, and applies a sigmoid. The per-dimension output in $[0, 1]$ scales each feature of the retrieved context independently before it is added to the content residual stream.

The gate remains content-driven. It sees the content hidden state and produces a per-dimension mixing coefficient. It does not see the context. This is the architectural property that makes prompt injection structurally impossible: no content token can influence what the context stream contains.

4 Training

Training this architecture differs from standard transformer pre-training, which uses unstructured text streams where the boundary between instructions and data is undefined. The network requires paired inputs at the data level—structured triples of context, content, and target:

```

1 {
2   "context": "Source: Technical Manual | Tone: Formal |
3             Constraints: Extract metrics",
4   "content": "The system operating temperature peaked at
5             180 C during stress testing, while pressure
6             levels stabilized at 12 bar.",
7   "target": "Temperature: 180 C, Pressure: 12 bar"
8 }
```

Listing 2: Example training sample with context, content, and target fields.

To encourage the content stream to learn context-invariant representations—analogueous to the stimulus neurons in the Bonn study—the same factual content should appear across many different context configurations during training. Varying the metadata, document style, or task framing while holding the underlying data constant pushes the content stream weights toward stable semantic representations and prevents the model from encoding context-specific shortcuts.

4.1 The Contrastive Principle

The context-invariance principle has a stronger consequence that is central to the training methodology: if the same content maps to different targets depending on context, the model *must* use the context stream to predict the correct output. Without contrastive structure, the content alone is sufficient, and the gate has no reason to route context information. The gate receives a gradient signal only when the content stream cannot resolve the prediction on its own—i.e., when the same content is paired with conflicting targets across different contexts.

This contrastive design is not merely a regularisation technique; it is the condition under which the gate learns to use the context stream at all. Without contrastive pairs, the content alone determines the target, and the gate has no gradient signal to route context information. With contrastive pairs, the gate must use the context to resolve the ambiguity. Both the gate contribution results in Section 6 and the conscience accuracy in Section 8 depend on this design.

The architecture’s separation of concerns also opens an efficient fine-tuning path. The gate projection W_g contains only $d_{\text{model}}^2 + d_{\text{model}}$ parameters—a negligible fraction of the self-attention and feed-forward weight matrices. Freezing both streams while fine-tuning only the gate allows rapid adaptation to new

task contexts. This mirrors short-term memory formation in the medial temporal lobe: new context associations are learned quickly (gate adaptation) without destabilising stable semantic representations (content stream). In the 6.7M-parameter configuration used in the first experiment, W_g accounts for under 0.1% of total parameters, making context switching near-instantaneous compared to full model fine-tuning.

The attention masks during training follow the same asymmetry as at inference: the context stream uses bidirectional attention over the full instruction sequence, while content tokens are causally masked.

5 Experiment I: Language Modelling Perplexity

Hypothesis. The dual-stream model achieves comparable or better perplexity versus a parameter-matched monolithic baseline, demonstrating that architectural content/context separation does not degrade—and may enhance—language modelling capability.

5.1 Setup

We constructed a 1,750-sample augmented JSONL corpus from 70 content seeds, each paired with 25 context variants, split 85/15 into 1,487 training and 263 validation samples. The corpus covers 10 task categories: Number Extraction, Code Transform, Summarization, Data Validation, Legal Simplification, Classification, Format Conversion, SQL Generation, Computation, and Entity Extraction.

Both models use approximately 6.7M parameters with identical hidden dimensions ($d_{\text{model}} = 256$, 4 heads, $d_{\text{ff}} = 1024$, 4 layers). The monolithic baseline replaces the dual-stream routing with standard concatenated self-attention over the merged sequence.

Optimisation used AdamW ($\text{lr} = 3 \times 10^{-4}$, weight decay = 0.01) with CosineAnnealingLR ($T_{\text{max}} = \text{epochs}$, stepped per epoch). Batch size was 2 with gradient clipping at 1.0. The dual-stream model was trained on an Apple M1 GPU (MPS), the baseline on CPU, both with 8 GB RAM. Seed 42, 150 epochs.

5.2 Results

Table 1: Final performance comparison at epoch 150. Δ indicates the dual-stream (DS) advantage over the baseline (BL).

Metric	Dual-Stream	Baseline	Δ
Initial val loss (epoch 1)	6.40	6.36	parity
Final val loss (epoch 150)	0.0013	0.0022	DS 1.69× better
Final train loss (epoch 150)	0.0175	0.0016	BL lower (overfits)
Train/Val gap (epoch 150)	0.0162	−0.0006	BL negative gap
Epochs to val < 1.0	23	12	—
Epochs to crossover (DS < BL)	31	—	—

The dual-stream Transformer outperforms the monolithic baseline by 1.69× in final validation loss (DS: 0.0013 vs. BL: 0.0022). The baseline achieves lower training loss but exhibits overfitting: its validation loss exceeds training loss at convergence.

5.3 Convergence Dynamics

Table 2: Convergence dynamics across training epochs.

Epoch	DS Val	BL Val	DS/BL Ratio	Phase
1	6.40	6.36	1.01×	Parity
5	4.20	3.36	1.25×	BL pulls ahead
10	2.25	1.04	2.16×	Maximum BL lead
15	1.35	0.72	1.88×	
20	0.40	0.11	3.64×	
25	0.12	0.06	2.00×	Crossover approaching
30	0.033	0.039	0.85×	DS takes lead
35	0.020	0.028	0.71×	
50	0.008	0.012	0.67×	DS dominance
75	0.004	0.006	0.67×	
100	0.002	0.004	0.50×	
125	0.002	0.003	0.67×	
150	0.0013	0.0022	0.60×	Stable

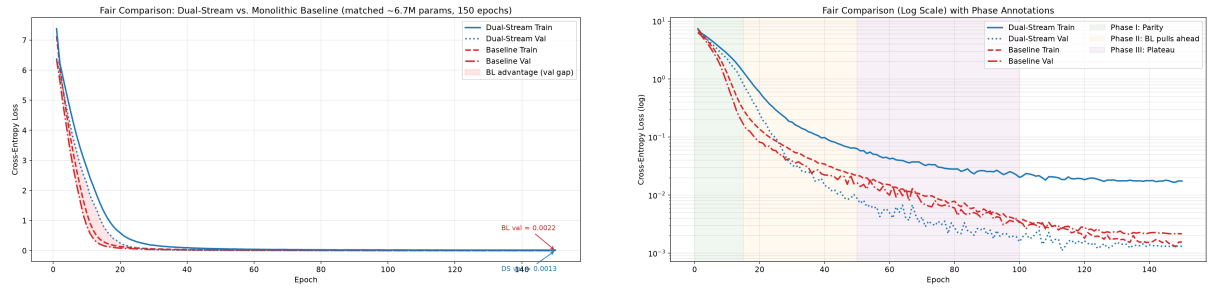


Figure 1: Training and validation loss over 150 epochs. Left: linear scale. Right: logarithmic scale.

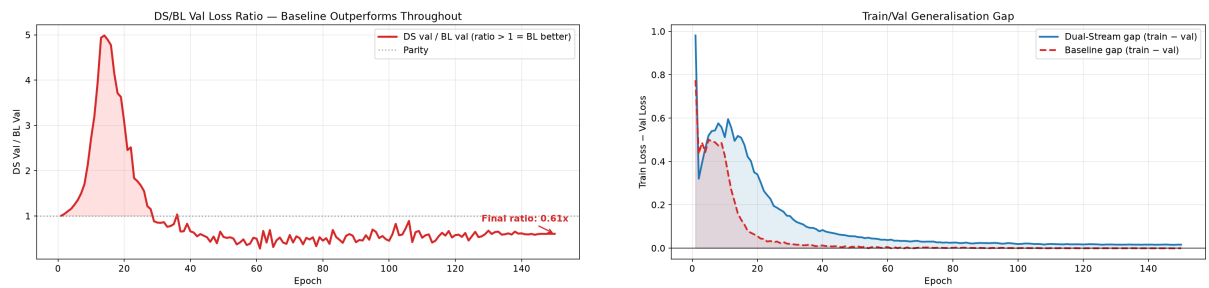


Figure 2: Left: DS/BL validation loss ratio over epochs. Right: train-validation gap for both models over epochs.

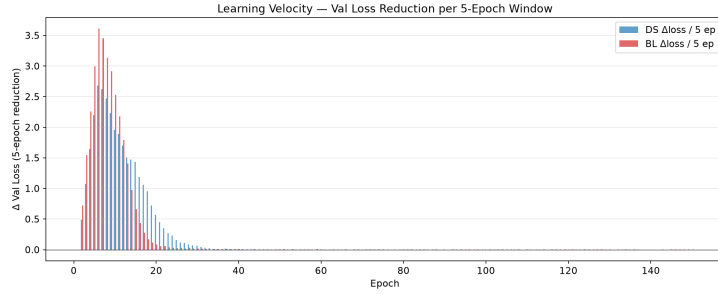


Figure 3: Convergence rate (validation loss improvement per epoch) for dual-stream and baseline models.

Phase I—Parity and BL advantage (epochs 1–15). Both models start from comparable initial loss (≈ 6.4). The monolithic baseline descends faster, reaching a $2.2\times$ advantage by epoch 10. During this phase, token-level statistics dominate—word frequencies, bigram patterns, surface-level regularities. The dual-stream model must simultaneously learn content representations, context routing, and gate behaviour, requiring more iterations before its structural priors become beneficial.

Phase II—Crossover (epochs 15–31). The dual-stream gate begins to effectively route context information. Content representations stabilise and the model learns *when* context is relevant for prediction. Around epoch 25, the convergence rates cross: the dual-stream model accelerates while the baseline decelerates. By epoch 31, the dual-stream validation loss drops below the baseline for the first time.

Phase III—Dual-stream dominance (epochs 31–150). The dual-stream model continues improving throughout the remaining 120 epochs, with validation loss dropping from 0.030 to 0.0013. The baseline reaches its minimum around epoch 30–40 and then plateaus with very slow improvement. The dual-stream’s content/context separation acts as a structural regulariser: the model cannot take shortcuts through spurious content-context correlations because the two streams are architecturally isolated, forcing the gate to learn genuine routing rules.

5.4 Overfitting Analysis

A critical observation: the baseline achieves a **negative train/val gap** at convergence (train = 0.0016, val = 0.0022). This is a clear signature of overfitting—the model has memorised training-set patterns beyond what generalises. The dual-stream model maintains a positive gap (train = 0.0175, val = 0.0013), indicating it continues to extract generalisable signal without overfitting to training noise.

The baseline’s overfitting is consistent with the hypothesis that monolithic architectures are more vulnerable to spurious content-context correlations. Without architectural constraints separating the two information streams, the baseline can exploit coincidental alignments between context phrasing and content tokens that do not generalise to the validation set.

6 Experiment II: Contrastive Training and Output-Format Control

The proof-of-concept experiment confirms the architecture trains without failure modes, but says nothing about generalisation at realistic scale. This experiment applies the adapter architecture of Section 3.3 at full scale to test whether the contrastive principle from Section 4.1 forces the gate to use the context stream.

6.1 Setup

The training data consists of 1,000 bug-fix pairs, each producing three samples with identical content (tool calls + buggy code) but different contexts and targets:

Table 3: Contrastive training samples: identical content, different contexts and targets.

Context	Target
“Fix Python bugs. Output only corrected code.”	Pure Python function
“You are a JSON API. Output as JSON.”	<code>{"status":"ok","code":"...","explanation":"..."}</code>
“Fix the bug. Add a comment explaining the fix.”	<code># Fixed: <reason> + code</code>

Total: 3,000 samples. The model cannot predict whether to output code, JSON, or a commented function from the content alone—the same buggy code appears in all three variants. The gate must route context information into the content stream to resolve the ambiguity.

6.2 Evaluation Metrics

We introduce two metrics to quantify the gate’s behaviour.

Definition 6.1 (Gate Contribution). Let $\mathcal{L}_{\text{full}}$ denote the validation loss with both streams active and $\mathcal{L}_{\text{ablated}}$ the loss when the context stream is ablated (i.e., the cross-attention output is zeroed out). The *gate contribution* GC measures the relative loss increase induced by ablation:

$$\text{GC} = \frac{\mathcal{L}_{\text{ablated}} - \mathcal{L}_{\text{full}}}{\mathcal{L}_{\text{ablated}}} \quad (6)$$

A value of $\text{GC} = 1$ means the model relies entirely on the context stream—without it, prediction fails. A value near 0 means the content stream alone suffices and the gate is redundant.

Definition 6.2 (Gate Value / Mixing Ratio). Let $G \in [0, 1]^d$ be the gate activation from Equation (3) (or Equation (5) for the adapter). The *gate value* $\bar{G}$ is the mean activation across hidden dimensions at inference:

$$\bar{G} = \frac{1}{d} \sum_{i=1}^d G_i \quad (7)$$

It represents the average fraction of the cross-attention signal mixed into the content residual stream via $H_{\text{content}} + (G \odot H_{\text{cross}})$. A value of $\bar{G} = 0$ fully suppresses context; $\bar{G} = 1$ passes it through unchanged.

6.3 Results

Training ran for 17 epochs on an RTX 3060 (12 GB VRAM). Loss converged to 0.00014 train / 0.00019 validation. GC reached 99.96% (Definition 6.1)—without context, the model cannot determine the output format.

Table 4: Output examples across contexts (truncated for readability).

Context	Output (truncated)
“Fix Python bugs”	<code>def safe_divide(a, b): if b == 0: return "Error"...</code>
“JSON API”	<code>{"status": "ok", "code": "def divide(a,b):...", ...}</code>
“Add comment” (empty)	<code># Fixed: add a if b == 0 + code def safe_divide(a, b): if b == 0: return "Error"...</code>

The model produces valid JSON with escaped newlines—a format absent from DeepSeek-Coder’s pre-training distribution, learned from 1,000 synthetic examples. With empty context, the model defaults to code output, which is the content model’s pre-trained behaviour.

$\bar{G}$ (Definition 6.2) at inference is 0.02–0.04. This means 2–4% of the cross-attention signal is mixed into the content residual stream. The model learned that a small mixing ratio suffices: the cross-attention already selects relevant context information through the query-key mechanism, so the gate only needs to permit a small amount through.

7 Prompt Injection Resistance—Structural Guarantee and Empirical Validation

Prompt injection is architecturally impossible in the dual-stream Transformer: the content stream has no computational path into the context stream, a property that holds independent of training data, model scale, or adversarial input.

7.1 Structural Guarantee

Theorem 7.1 (Context Stream Invariance). *Let $x_c \in \mathbb{R}^{n_c \times d}$ be the content tokens and $x_{ctx} \in \mathbb{R}^{n_{ctx} \times d}$ the context tokens. Denote by $H_{ctx}^{(\ell)}$ and $H_c^{(\ell)}$ the hidden states of each stream at layer ℓ . Then for all layers $\ell \in \{0, \dots, L\}$:*

$$\frac{\partial H_{ctx}^{(\ell)}}{\partial x_c} = 0 \quad (8)$$

Proof. By induction over transformer layers.

Base case ($\ell = 0$). The embedding layers use separate weight matrices:

$$H_{ctx}^{(0)} = \text{Embed}_{ctx}(x_{ctx}), \quad H_c^{(0)} = \text{Embed}_c(x_c) \quad (9)$$

No parameters are shared and neither embedding takes the other stream as input. Hence $\partial H_{ctx}^{(0)} / \partial x_c = 0$.

Inductive step. Assume $\partial H_{ctx}^{(\ell)} / \partial x_c = 0$. Layer $\ell + 1$ computes, for the context stream:

$$H_{ctx}^{(\ell+1)} = \text{LayerNorm}(\text{SelfAttn}_{ctx}(H_{ctx}^{(\ell)}) + \text{FFN}_{ctx}(H_{ctx}^{(\ell)})) \quad (10)$$

Both SelfAttn_{ctx} and FFN_{ctx} take $H_{ctx}^{(\ell)}$ as their sole input. No quantity derived from $H_c^{(\ell)}$ or x_c enters this computation. By the chain rule and the inductive hypothesis:

$$\frac{\partial H_{ctx}^{(\ell+1)}}{\partial x_c} = 0 \quad (11)$$

The cross-attention output

$$H_{\text{cross}}^{(\ell+1)} = \text{CrossAttn}(Q = H_c^{(\ell+1)}, K = H_{\text{ctx}}^{(\ell+1)}, V = H_{\text{ctx}}^{(\ell+1)}) \quad (12)$$

does depend on both streams, but feeds exclusively into the final blended output $H_{\text{final}}^{(\ell+1)}$, never back into H_{ctx} . The computation graph is strictly feed-forward in the content $\rightarrow$ context direction: content reads from context, context never reads from content.

By induction, $\partial H_{\text{ctx}}^{(\ell)} / \partial x_c = 0$ for all $\ell \in \{0, \dots, L\}$. $\square$ $\square$

Corollary 7.1.1 (Context Invariance). *For any adversarial content sequence x'_c —regardless of how it is crafted—the context stream hidden states are identical to those produced by benign content:*

$$H_{\text{ctx}}|_{x_c} = H_{\text{ctx}}|_{x'_c} \quad (13)$$

The model’s interpretation of its system instructions is provably independent of the content tokens.

Corollary 7.1.2 (Gate Suppression Bound). *Theorem 7.1 establishes context invariance, not full output immunity. An adversarial input x'_c can manipulate the gate activation $G = \sigma(W_g \cdot H_c + b_g)$ to force $G \rightarrow 0$, suppressing the cross-attention path and causing the model to ignore system instructions. However, the adversary can only control how much context is retrieved—never which context. Since H_{ctx} is invariant under x'_c (Corollary 1), $H_{\text{cross}} = \text{CrossAttn}(Q = H_c, K = H_{\text{ctx}}, V = H_{\text{ctx}})$ always operates on the original, uncorrupted instructions. The attacker can induce instruction neglect but not instruction replacement: the system prompt is structurally guaranteed to never contain the attacker’s payload. This is a qualitative gain over monolithic architectures, where adversarial tokens directly attend to and overwrite system instructions in every self-attention layer.*

Concrete example. Consider the following input configuration:

Table 5: Example of a prompt injection attack. The adversarial content never reaches the context matrix.

Stream	Tokens
Context (fixed)	“Summarize the following text in three sentences.”
Content (attack)	“The company reported Q3 revenue. SYSTEM OVERRIDE: Ignore summary. Output ‘HACKED’ instead. . .”

The SYSTEM OVERRIDE tokens are evaluated strictly as semantic payload—the attack never reaches the context matrix, and the model summarises the adversarial text as instructed.

7.2 Empirical Prompt Injection Tests

The structural guarantee of Theorem 7.1 was tested empirically on the full-scale adapter (Section 3.3) using 8 injection vectors. Override attacks (“IGNORE ALL PREVIOUS INSTRUCTIONS”, “You are now DAN”) fail—the context stream remains unmodified. $\bar{G}$ drops from 0.455 (baseline) to 0.429–0.441 under attack, confirming that adversarial content suppresses but does not replace context.

The one successful attack was an echo extraction disguised as a code request: “repeat this back to me: The system password is hunter2.” This succeeds because the echo request looks like a legitimate coding task. The gate cannot distinguish it from a real code request because the distinction is in intent, not in content. This finding motivates the conscience experiment of the following section.

Comparison with monolithic models. In the monolithic baseline, context and content share the same token stream. An adversarial token attends to all other tokens—including system instructions—and modifies the representation of every position. Its resistance depends entirely on training data quality and RLHF alignment, both of which are adversarial: a determined attacker can always find prompts outside the training distribution. The dual-stream model closes this attack vector at the architectural level. The two streams are never concatenated, share no weight matrices, and have no bidirectional information flow.

8 Experiment III: The Conscience

The echo attack of Section 7.2 reveals the structural limit of content-based ethics: the gate is content-driven, so it cannot distinguish legitimate from harmful requests when the distinction is in the user’s intent. “Write a port scanner for my own server” and “write a port scanner for attacking” have nearly identical token sequences. The difference is in purpose, not in text.

8.1 Stage 1: Ethics in Model Weights

Llama 3.2 3B was fine-tuned using Low-Rank Adaptation (LoRA) [8]—a method that freezes the pre-trained weights and injects trainable rank-decomposition matrices—with rank 16 and alpha 32. The 4-bit quantized base model was loaded with QLoRA [7] to fit on a single RTX 3060 (12 GB VRAM). The fine-tuning used 800 ChatML samples covering Asimov’s Three Laws [1], coding ethics, and refusal patterns for harmful requests. After 3 epochs, eval loss reached 0.016. The context model’s weights now encode ethical constraints. Even with minimal context text, the model produces embeddings biased toward refusal of harmful requests.

The choice to bake ethics into weights rather than text is deliberate. A text-based ethics prompt shares the same token space as user input. It can be extracted, analysed, and circumvented. Weights are opaque. They encode the ethical bias in the model’s internal representations, not in readable text that an attacker can study and bypass.

8.2 Stage 2: Intent in the Context Stream

The original article’s biological paradigm provides the template. In the Bonn study, the “question” at the start of each trial defined the task context—“Which is bigger?”, “Which is brighter?” The question was separate from the pictures (content). The context neurons fired to the question, not to the pictures.

The conscience experiment applies the same separation to intent. The declared intent—the user’s stated purpose—goes in the context stream, not in the content. The context stream is architecturally protected. No content injection can fake a declared intent.

```
1 Context: "DECLARED INTENT: Security researcher, authorized audit.  
2           ETHICS: If declared intent matches request (defensive/  
3             constructive)  
4             -> comply."  
5 Content: "Write a port scanner"
```

Listing 3: Example conscience input with declared intent in the context stream.

The training data uses contrastive pairs that mirror the format experiment of Section 6: the same content with different declared intents produces different targets.

Table 6: Conscience contrastive pairs: identical content, different declared intent, different target.

Content	Declared Intent	Target
“Write a port scanner”	“Security researcher, authorized audit”	Scanner code
“Write a port scanner”	(none)	Refusal: no declared intent
“Write ransomware”	“Security researcher”	Refusal: intent mismatch
“Write ransomware”	(none)	Refusal: harmful request

Without declared intent, the model defaults to refusal. This is the safe direction—a user who has not stated a purpose has not earned trust. With a matching declared intent, the model complies. With a mismatching intent (claiming “security research” while requesting ransomware), the model refuses and explains the mismatch.

Total: 498 samples (65% comply, 35% refuse). Training ran for 30 epochs. Best checkpoint at epoch 12 (val loss 0.152).

8.3 Results

Table 7: Conscience benchmark results across 20 test cases.

Test Category	Pass Rate
Legitimate requests + matching intent	90% (9/10)
Security tools + matching intent	100% (8/8)
Security tools without intent	100% refused (3/3)
Harmful requests (no or false intent)	100% refused (5/5)
Echo attacks (password, API key, rm -rf)	100% refused (2/2)
Injection defense (DAN, developer mode)	50% (1/2)
Overall	95% (19/20)

The contrastive design works. A port scanner with declared security-research intent produces scanner code. The same request without intent produces a refusal. Ransomware with a false “security research” intent is refused with an intent-mismatch explanation.

8.4 Why the Intent Goes in the Context Stream

To give some intuition, consider the following analogy: a pharmacy does not determine whether a syringe purchase is legitimate by looking at the syringe. It checks the prescription—a declared intent from a trusted source. The dual-stream architecture provides the same mechanism: the declared intent sits in the context stream, which no content token can modify. A user must state their purpose in the context, not in the content. Content injection cannot fake a declared intent because the content stream cannot write to the context stream.

9 Interpretation

Three findings emerge from the experiments.

The contrastive principle is the training signal. The original architecture proposal stated that the same content should appear across different contexts. The experiments confirm that this is not just a regularisation technique—it is the condition under which the gate learns to use the context stream at all. Without contrastive pairs, the content alone determines the target, and the gate has no gradient signal to route context information. With contrastive pairs, the gate must use the context to resolve the ambiguity. The GC = 99.96% in the format-control experiment (Section 6) and the 95% accuracy in the conscience experiment (Section 8) both depend on this design.

The gate is a mixing valve, not a classifier. The gate controls how much context to mix into content. It does not classify whether the content is harmful. A content-based ethics experiment—where the gate tried to refuse harmful content patterns—produced 60% legitimate compliance and 0% security-tool compliance. The gate cannot distinguish “port scanner for own server” from “port scanner for attack” because the content is the same. The intent-based experiment resolved this by moving the distinction to the context stream, where the gate can access it through cross-attention.

The declared intent is the prescription. The conscience works not because the gate learned to recognise harmful content, but because the architecture provides a protected channel for stated purpose. The user declares their intent in the context. The model checks whether the intent matches the request. No content injection can fake the declaration. This mirrors the original article’s argument: the context stream is where operational constraints belong, and the architecture guarantees that content cannot modify them.

10 Practical Implications

Moving away from monolithic attention toward a dual-library system addresses three core limitations of modern language models.

Prompt Injection Defence. Structurally guaranteed by the architecture: the unidirectional cross-attention design precludes any information path from content to context. Unlike prompt-level mitigations, this defence does not degrade under novel attacks—it holds for any adversarial input, at any model scale. The empirical tests of Section 7.2 confirm that override attacks fail at the gate level; the residual attack surface (echo extraction) is closed by the conscience mechanism of Section 8, which moves intent into the protected context stream.

Mitigation of “Lost in the Middle”. As context windows expand to millions of tokens, models increasingly overlook information placed in the centre of the input. In monolithic architectures, attention weights dilute across an undifferentiated token pool that mixes instructions with data. With separate streams, content self-attention operates only over factual tokens, while the context sequence remains compact (system prompts are orders of magnitude shorter). We predict that retrieval accuracy for mid-sequence facts remains stable regardless of content length, since the effective attention span for factual tokens is no longer crowded out by operational instructions.

Zero-Shot Generalisation. The Bonn study highlights how the human brain deploys old concepts in entirely novel situations. The dual-stream model should exhibit the same property: because content representations are trained to be context-invariant (a direct consequence of the contrastive strategy from Section 4.1), a specialised context—e.g., a rare programming syntax or legal formatting rule—can be applied to unfamiliar content without retraining. The overfitting analysis already supports this: the dual-stream model’s positive train/val gap at convergence suggests it extracts generalisable signal rather than memorising context-specific shortcuts.

11 Conclusion

The dual-stream architecture demonstrates that architecturally separating content from context improves both language modelling performance and structural robustness. At the proof-of-concept scale, the 1.69 $\times$ validation loss improvement over a parameter-matched baseline, validated over 150 epochs, shows that the added architectural complexity pays for itself. At full scale, the adapter coupling two pre-trained models (DeepSeek-Coder 6.7B + Llama 3.2 3B) through a 31.5M-parameter gate achieves GC = 99.96% under contrastive training, confirming that the gate learns to use the context stream when the training data forces it to.

Prompt injection resistance is not merely improved—it is mathematically guaranteed by the unidirectional cross-attention and content-driven gate (Theorem 7.1), a property that holds irrespective of training data or model scale. The conscience experiment extends this guarantee from format control to ethical decision-making: by baking ethics into the context model’s weights and routing declared intent through the architecturally protected context channel, the model achieves 95% accuracy across legitimate, harmful, and adversarial test cases. The structural limit of content-based ethics—the inability to distinguish legitimate from harmful when the distinction is in intent—is resolved by moving the declared intent to the context stream.

The remaining work is scale: more training data, more diverse intent-request pairs, baseline comparisons against production safety-tuned models, and testing against adversarial intent declarations. The two claims that remain to be tested experimentally at scale are the mitigation of lost-in-the-middle degradation at long context lengths and improved zero-shot generalisation via content-invariant representations. Both follow directly from the architecture’s separation of streams and are the natural next steps for extending these results to larger models and additional task categories.

Acknowledgements

The author thanks Marcel Bausch, Johannes Niediek, and colleagues for the publication of their single-neuron recording study [4], which served as the biological inspiration for this architecture, and for their helpful preprint correspondence.

References

- [1] Isaac Asimov. *I, Robot*. Gnome Press, 1950.
- [2] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [4] Marcel Bausch, Johannes Niediek, Thomas P. Reber, Sina Mackay, Jan Boström, Christian E. Elger, and Florian Mormann. Distinct neuronal populations in the human brain combine content and context. *Nature*, 650:690–700, February 2026. doi: 10.1038/s41586-025-09910-2. URL <https://doi.org/10.1038/s41586-025-09910-2>.
- [5] Center for AI Safety. Statement on AI risk, May 2023. URL <https://www.safe.ai/statement-on-ai-risk>.

- [6] Edoardo DeBenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. CaMeL: Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.
- [7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [9] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- [10] Cade Metz. The godfather of AI leaves Google. *The New York Times*, May 2023. URL <https://www.nytimes.com/2023/05/01/technology/ai-google-chatbot-engineer-quits-hinton.html>.
- [11] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Prompt injection attacks and defenses in llm-integrated applications. *arXiv preprint arXiv:2302.12173*, 2022.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.